

Extension methods in C#

Extension methods allow you to add new methods to existing types without modifying their original source code, creating a derived type, or recompiling them. They are static methods that act like instance methods. You can call them using standard dot-notation (object.Method()) directly on the extended type.

Core Rules for Implementation

To build an extension method, you must follow these rules:

- **Static Class:** The containing class must be a non-nested, non-generic static class.
- **Static Method:** The method itself must be declared with the static keyword.
- **The this Modifier:** The very first parameter specifies the target type and must be prefixed with this.
- **Namespace Scope:** The method is only visible when its namespace is explicitly imported via a using directive.

The code block below demonstrates how to extend the built-in string type to add few extension methods.

```
using System.Text.RegularExpressions;

namespace ExtnMethodDemo
{
    public class Program
    {
        public static void Main()
        {
            string text = "C# extension methods are highly useful.";

            // Called like an ordinary instance method
            // --- Testing WordCount ---
            int count = text.WordCount();
            Console.WriteLine($"Word count: {count}"); // Output: 6

            // --- Testing Truncate ---
            string longTitle = "Advanced C# Design Patterns and Architecture";
            string shortTitle = longTitle.Truncate(15);
            Console.WriteLine(shortTitle);
            // Output: Advanced C# Des...

            // --- Testing IsValidEmail ---
            string userEmail = "test@example.com";
            string badEmail = "invalid-email@com";

            Console.WriteLine(userEmail.IsValidEmail()); // Output: True
            Console.WriteLine(badEmail.IsValidEmail()); // Output: False
        }
    }
}
```

Free, Open-Source, Cross-Platform Framework

```
// 1. Must be a static class
// 2. Must be a static method
// 3. First parameter uses 'this' to target the string type
public static class StringExtensions
{
    // Example 1: Counts the number of words in a string
    public static int WordCount(this string str)
    {
        if (string.IsNullOrEmpty(str))
            return 0;

        string[] words = str.Split(new[] { ' ', '\t', '\n' },
StringSplitOptions.RemoveEmptyEntries);
        return words.Length;
    }
    // Example 2: Validates if the string matches a basic email format
    public static bool IsValidEmail(this string input)
    {
        if (string.IsNullOrEmpty(input))
            return false;

        // Simple regex check for demonstration
        string pattern = @"^[^@\s]+@[^@\s]+\.[^@\s]+$";
        return Regex.IsMatch(input, pattern, RegexOptions.IgnoreCase);
    }
    // Example 3: Truncates a string and appends a suffix (like "...") if it
    exceeds a limit
    public static string Truncate(this string input, int maxLength, string
suffix = "...")
    {
        if (string.IsNullOrEmpty(input) || input.Length <= maxLength)
            return input;

        return input.Substring(0, maxLength) + suffix;
    }
}
}
```

☪*田 Enjoy Programming 田*☪